

Software-Assignment: Image Compression Using Singular Value Decomposition

Harichandana Varanasi-ai25btech11039

*

1 Aim

The aim of this project is to implement **image compression using truncated Singular Value Decomposition (SVD)** in C. The objective is to reconstruct grayscale images with reduced storage requirements by retaining only the top k singular values, and to analyze the effect of k on image quality and compression efficiency.

2 Introduction

A grayscale image can be represented as a matrix $A \in R^{m \times n}$, where each element corresponds to a pixel intensity between 0 and 255. Using Singular Value Decomposition, the matrix can be factorized as:

$$A = U\Sigma V^T$$

where U and V are orthogonal matrices, and Σ is a diagonal matrix containing singular values in decreasing order. By keeping only the top k singular values and corresponding vectors, we can form a low-rank approximation:

$$A_k = U_k \Sigma_k V_k^T$$

which preserves most of the important image information while reducing data size. This concept forms the mathematical basis for image compression. In this project, the entire process is implemented in C by reading and writing `.pgm` images, performing SVD, reconstructing the images for various k values, and computing the compression error using the Frobenius norm.

3 Summary of Gilbert Strang's Video on SVD

The Singular Value Decomposition (SVD) is a fundamental matrix factorization method which expresses any matrix A as:

$$A = U\Sigma V^T$$

where:

- U is an orthogonal matrix whose columns are the **left singular vectors**.
- Σ is a diagonal matrix containing the **singular values** (positive numbers) on its diagonal.
- V^T is the transpose of an orthogonal matrix V , whose columns are the **right singular vectors**.

a) Computation of U , Σ , and V

To compute the SVD, we evaluate the matrices $A^T A$ and AA^T :

$$A^T A = V \Sigma^2 V^T \quad \text{and} \quad AA^T = U \Sigma^2 U^T$$

- $A^T A$ and AA^T are symmetric, positive semi-definite matrices.
- The eigenvectors of $A^T A$ form the columns of V .
- The eigenvectors of AA^T form the columns of U .
- The eigenvalues of $A^T A$ (and AA^T) are the squares of the singular values in Σ .

b) Applications

The SVD is widely used in data analysis and statistics, especially for understanding large data matrices. It decomposes a matrix into components where the singular values indicate the importance of each component. The triplet (U, Σ, V^T) represents the principal components of the data, with the largest singular value σ_1 corresponding to the most significant direction of variation.

4 Theory / Background

A grayscale image can be represented as a two-dimensional matrix $A \in R^{m \times n}$, where each element a_{ij} corresponds to the pixel intensity value (0 for black and 255 for white). The goal of image compression is to represent this matrix using fewer numbers while preserving as much visual information as possible.

Singular Value Decomposition (SVD)

The Singular Value Decomposition (SVD) of a matrix A is given by:

$$A = U \Sigma V^T$$

where:

- U is an $m \times m$ orthogonal matrix whose columns are called *left singular vectors*.
- Σ is an $m \times n$ diagonal matrix whose diagonal elements $\sigma_1, \sigma_2, \dots$ are called *singular values* of A .
- V is an $n \times n$ orthogonal matrix whose columns are *right singular vectors*.

The singular values are always non-negative and are arranged in decreasing order:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$$

where r is the rank of A .

Low-Rank Approximation

Most of the image information is contained in the first few singular values. Hence, we can approximate A by keeping only the top k singular values and corresponding vectors:

$$A_k = U_k \Sigma_k V_k^T$$

where:

- U_k contains the first k columns of U ,
- Σ_k is the $k \times k$ diagonal matrix with top k singular values,
- V_k contains the first k columns of V .

The matrix A_k is called the *rank- k approximation* of A and it provides the best approximation of A among all rank- k matrices in the Frobenius norm sense.

Application to Image Compression

In image compression using SVD:

1. Read the image and represent it as a matrix A .
2. Compute its SVD to obtain U , Σ , and V^T .
3. Choose a value of k (e.g., 5, 20, 50, 100).
4. Form the rank- k approximation $A_k = U_k \Sigma_k V_k^T$.
5. Reconstruct and store the compressed image from A_k .

This method allows storing only the first k singular values and corresponding columns of U and V , leading to a significant reduction in data size while maintaining acceptable image quality.

5 Algorithm Selection and Implementation

Algorithm Comparison

Several algorithms exist for computing the Singular Value Decomposition (SVD):

1. **Jacobi Method (Chosen Method):**

- Uses iterative rotations to diagonalize $A^T A$
- Highly accurate but computationally expensive
- Simple to implement for smaller matrices

2. **QR Algorithm:**

- First reduces the matrix to bidiagonal form, then applies QR iterations
- Standard method in numerical libraries (LAPACK)

3. **Power Iteration:**

- Iteratively finds dominant eigenvectors
- Efficient for finding top- k singular values (partial SVD)

4. **Divide and Conquer:**

- Fastest for dense matrices
- Complex implementation

Pseudocode

Algorithm: Jacobi SVD Image Compression

Input:

- Input image file path (P2 .pgm format)
- Output image file path for reconstructed image
- k — number of singular values to retain

Steps:

1. **Read the input image:**

- Open the .pgm file and verify it is in P2 format.
- Ensure size = 256×256 .
- Store pixel values in matrix A (as double).

2. **Compute SVD using Jacobi method:**

- Transpose A to get A^T .

- Compute $A^T A$.
- Apply Jacobi method on $A^T A$ to obtain:
 - Eigenvalues (singular values squared)
 - Eigenvectors (V matrix)
- Sort eigenvalues in descending order and reorder V accordingly.
- Compute singular values $S = \sqrt{\text{eigenvalues}}$.
- Compute $U = (A \times V)/S$ for each column.

3. **Reconstruct the image:**

- Initialize result matrix with zeros.
- For $i = 1$ to k :

$$result += U[:, i] \times S[i] \times V[:, i]^T$$

4. **Compute Frobenius reconstruction error:**

$$error = \sqrt{\sum (A - result)^2}$$

5. **Convert** reconstructed matrix to valid pixel range (0–255).
6. **Write** reconstructed image to output (.pgm) file.

Output:

- Reconstructed image file
- Printed reconstruction error

]

6 Results and Discussion

Reconstructed Images

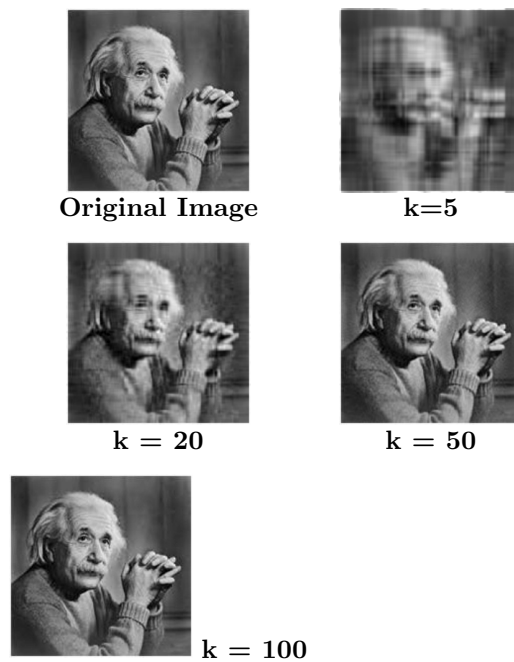


Figure 1: Comparison of reconstructed images at different k -values

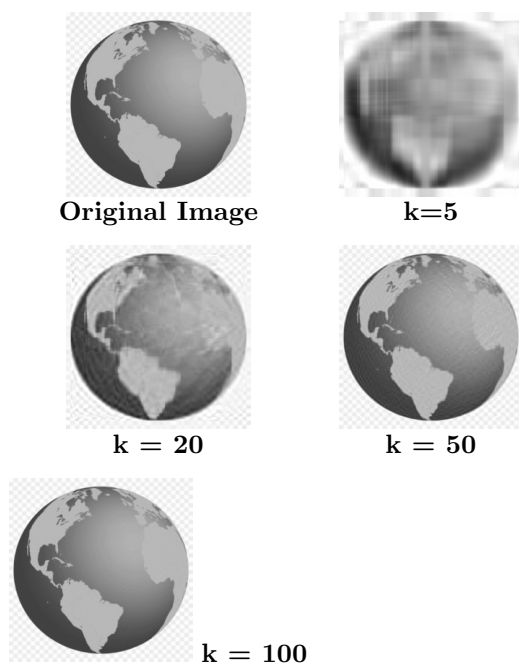


Figure 2: Comparison of reconstructed images at different k -values

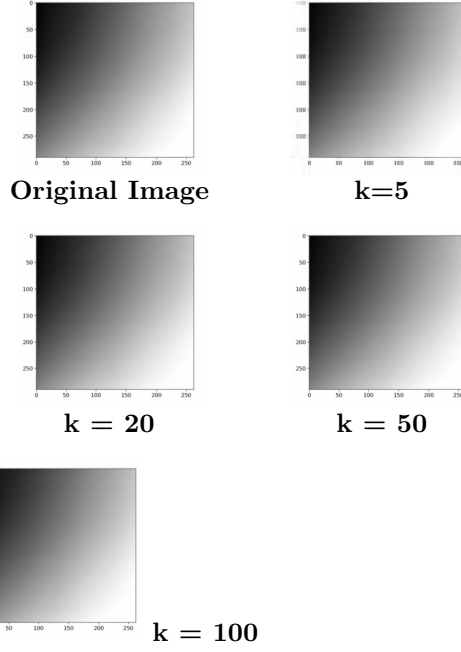


Figure 3: Comparison of reconstructed images at different k -values

Frobenius Norm Error

The Frobenius norm error measures the difference between the original image matrix A and its rank- k approximation A_k . It quantifies the reconstruction accuracy of the compressed image and is defined as:

$$\|A - A_k\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n (a_{ij} - (A_k)_{ij})^2} = \sqrt{\sum_{i=k+1}^r \sigma_i^2}$$

where σ_i are the singular values of A . A smaller $\|A - A_k\|_F$ indicates a more accurate reconstruction of the original image.

Error Analysis

The following tables show the Frobenius norm error $\|A - A_k\|_F$ for different images at various ranks k .

k (Rank)	Frobenius Error
5	6160.223726
20	2504.727435
50	872.667241
100	136.801124

Table 1: Error analysis for Image 1 (Einstein) at different k values.

k (Rank)	Frobenius Error
5	5746.125790
20	2531.968656
50	1085.598686
100	358.585450

Table 2: Error analysis for Image 2 (Globe) at different k values.

k (Rank)	Frobenius Error
5	1354.389000
20	176.341132
50	55.868794
100	32.221596

Table 3: Error analysis for Image 3 (Greyscale) at different k values.

7 Compression Trade-offs

For an $m \times n$ image:

- **Original:** mn values
- **Compressed (rank- k):** $k(m + n + 1)$ values

The **Compression Ratio (CR)** is given by:

$$CompressionRatio = \frac{mn}{k(m + n + 1)}$$

A higher compression ratio indicates more storage savings, but usually results in a loss of image quality. As k increases, the reconstructed image retains more singular values, improving the quality but requiring more storage. Thus, there exists a trade-off between compression efficiency and reconstruction accuracy.

Reflection (Option B, C-only). Full C gave control and reproducibility; trade-off was more coding effort and slower than optimized libraries. (Hybrid Option A eases experimentation/plots but adds cross-language overhead.)

8 Conclusion

The project demonstrates how Singular Value Decomposition (SVD) can be effectively used for image compression. By keeping only the dominant singular values, the image quality and file size can be balanced as needed. The results clearly show that smaller k values lead to higher compression, while larger k values give better visual quality. Overall, the work provided practical insight into applying linear algebra concepts for efficient image representation and storage.